

Spring Boot 설정·빌드 파일 완전 정리

pom.xml · build.gradle(.kts) · gradlew · application.properties · application.yml

1. 한눈에 보는 역할

파일	역할	빌드도구	핵심
pom.xml	의존성·플러그인·빌드 설정	Maven	XML 기반
build.gradle	의존성·플러그인·빌드 설정	Gradle	Groovy DSL
build.gradle.kts	의존성·플러그인·빌드 설정	Gradle	Kotlin DSL
gradlew / gradlew.bat	Gradle Wrapper 실행	Gradle	프로젝트 지정 Gradle 실행
application.properties	Spring Boot 실행환경 설정	공통	key=value
application.yml	Spring Boot 실행환경 설정	공통	계층형 YAML

핵심: pom.xml과 build.gradle(.kts)는 '빌드 설정', gradlew는 'Gradle 실행기', application.properties/yml은 '실행된 Spring Boot의 환경 설정'임.

2. pom.xml — Maven

POM(Project Object Model)은 Maven 프로젝트의 중심 파일임. Java 버전, Spring Boot 의존성, JPA, JDBC 드라이버, 테스트 및 빌드 플러그인을 정의함.

pom.xml 예제

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.mariadb.jdbc</groupId>
    <artifactId>mariadb-java-client</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

xml

Maven Wrapper 실행

```
Windows: .\mvnw.cmd spring-boot:run
Linux/macOS: ./mvnw spring-boot:run
```

text

3. Gradle과 build.gradle.kts

Gradle은 Maven과 같은 빌드 자동화 도구임. Kotlin + Spring Boot에서는 build.gradle.kts(Kotlin DSL)를 사용하면 언어 통일성과 IDE 지원 측면에서 편리함.

build.gradle.kts 예제

```
plugins {  
    java  
    id("org.springframework.boot") version "<Spring Boot 버전>"  
    id("io.spring.dependency-management") version "<플러그인 버전>"  
}  
repositories { mavenCentral() }  
dependencies {  
    implementation("org.springframework.boot:spring-boot-starter-web")  
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")  
    runtimeOnly("org.mariadb.jdbc:mariadb-java-client")  
}
```

kotlin

4. gradlew — Gradle Wrapper

gradlew는 설정 파일이 아니라 Gradle Wrapper 실행 스크립트임. 개발 PC에 Gradle을 별도로 설치하지 않아도 프로젝트가 지정한 Gradle 버전을 받아 동일한 환경으로 빌드할 수 있게 함.

정상 Wrapper 구조

```
project/  
├─ build.gradle.kts  
├─ settings.gradle.kts  
├─ gradlew  
├─ gradlew.bat  
└─ gradle/  
    └─ wrapper/  
        ├─ gradle-wrapper.jar  
        └─ gradle-wrapper.properties
```

text

실행 명령

```
Windows: .\gradlew bootRun  
Linux/macOS: ./gradlew bootRun
```

text

org.gradle.wrapper.GradleWrapperMain 오류가 나오면 gradle-wrapper.jar 등 Wrapper 구성 파일 누락 여부를 우선 확인함.

5. Maven vs Gradle 비교

항목	Maven	Gradle
핵심 파일	pom.xml	build.gradle / build.gradle.kts
문법	XML	Groovy 또는 Kotlin DSL
Wrapper	mvnw	gradlew
설정량	상대적으로 많음	상대적으로 간결

항목	Maven	Gradle
Java 프로젝트	매우 보편적	매우 보편적
Kotlin 프로젝트	가능	특히 잘 어울림
Spring Boot	강력 지원	강력 지원

6. application.properties

src/main/resources에 두며 Spring Boot의 서버 포트, DB 접속정보, JPA 옵션 등 런타임 환경을 설정함.

MariaDB properties 예제

```
spring.application.name=member
server.port=8080
spring.datasource.url=jdbc:mariadb://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=1234
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

properties

7. application.yml

properties와 목적은 같지만 계층 구조와 들여쓰기를 사용함. 설정이 많아질수록 관련 항목을 묶어 읽기 편함.

MariaDB YAML 예제

```
spring:
  application:
    name: member
  datasource:
    url: jdbc:mariadb://localhost:3306/testdb
    username: root
    password: 1234
    driver-class-name: org.mariadb.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  server:
    port: 8080
```

yaml

8. properties vs yaml 비교

항목	application.properties	application.yml
형식	key=value	계층형 YAML
들여쓰기	중요하지 않음	매우 중요
입문	매우 쉬움	쉬움
설정 많을 때	길어질 수 있음	구조적으로 정리하기 좋음
중복 표현	상대적으로 많음	적음

항목	application.properties	application.yml
Spring Boot 지원	지원	지원
추천	단순/학습용	중대형 설정

둘을 동시에 사용할 수도 있으나 학습 단계에서는 한 형식으로 통일하는 것이 설정 충돌과 혼동을 줄이는 데 좋음.

9. Oracle · PostgreSQL · MariaDB 설정 비교

DB	JDBC URL 예	Driver
MariaDB	jdbc:mariadb://localhost:3306/testdb	org.mariadb.jdbc.Driver
PostgreSQL	jdbc:postgresql://localhost:5432/testdb	org.postgresql.Driver
Oracle	jdbc:oracle:thin:@localhost:1521/FREEPDB1	oracle.jdbc.OracleDriver

공통 application.yml 형태

```
spring:
  datasource:
    url: <DB별 JDBC URL>
    username: <사용자>
    password: <비밀번호>
    driver-class-name: <DB별 Driver>
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

yml

10. Java / Kotlin 권장 구성

용도	권장 구성
Java Spring Boot 학습	Maven + pom.xml + application.yml
Kotlin Spring Boot 주력	Gradle + build.gradle.kts + gradlew + application.yml
DB 입문	MariaDB
데이터·AI 서비스	PostgreSQL도 적극 학습
기업·공공 환경	Oracle도 학습

권장 흐름: build.gradle.kts 또는 pom.xml → 의존성 준비 → gradlew/mvnw로 Spring Boot 실행 → application.yml(properties) 로드 → DB 연결 → Controller → Service → Repository → JPA → Database.

11. 최종 암기

pom.xml / build.gradle.kts = 프로그램을 어떻게 빌드할지 설정함.

gradlew = 프로젝트가 요구하는 Gradle을 실행함.

application.properties / application.yml = 실행된 Spring Boot가 어떤 포트·DB·JPA 설정으로 동작할지 지정함.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME...)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.